#6 Cycle through all edges in `graph`. Follows the same pattern as for vertices.
#7 If we made it all the way to this line, then `graph` is planar: we need to update the cache and return `true`.

There are other small improvements that we can add; although not as impactful as avoiding duplicates, they all contribute to speeding up the algorithm. For instance, we can improve our halting condition: we don't need to wait till we get down to K_5 or $K_{3,3}$, any complete graph with 5 or more vertices, or any complete bipartite graph with both partitions having 3 or more vertices, are certainly non-planar.

Most of these cases, however, are already caught by the Euler's invariants.

Listing 15.7 shows the pseudocode for the improved method; you can find a Java implementation on the book's repo, or a JavaScript implementation provided by JsGraph.

15.2.5 Efficient Algorithms

The algorithm presented in listing 15.7 is still too slow to be affordable on large graphs; still, it can be a feasible low-cost[16] option that can work on small graphs.

There are far better algorithms that have been developed for planarity testing; while they take different approaches to provide an answer (and a planar embedding) in linear time, they all have something in common: they are fairly complicated.